CENG-567
REINFORCEMENT LEARNING

Lecture-8

On- and Off-Policy approaches using Monte Carlo and Temporal Difference, Sarsa, Q-Learning

Dr. Osman GÖKALP

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Recap (Why Monte Carlo Methods?)

Monte Carlo (MC) approach

Learn from *sampled experience*, not from environment models.

Analogy: estimating a biased coin → flip many times instead of modeling physics.

Why MC in RL?

Real environments are often too complex or unknown.

No need for transition probabilities or rewards model.

MC Prediction

Use complete episodes to estimate:

State values $v_\pi(s)$

Action values $q_\pi(s, a)$

Values are estimated by *averaging returns* from observed trajectories.

As number of samples ↑ → estimates converge to true values.

Recap (MC Prediction: Key Concepts)

Using Trajectories

A trajectory: sequence of (state, action, reward).

Each visit to a state provides a sample return.

First-Visit vs Every-Visit MC

First-Visit MC: update only on the first occurrence of a state in each episode. Unbiased.

Every-Visit MC: use all occurrences.

- Biased for small samples, but bias → 0 with many visits.
- Naturally provides more data (useful for function approximation).

Limitations of MC Prediction

Needs *complete episodes*.

Cannot improve a policy using only state values (no q-values unless environment model is known).

Deterministic policies may not explore → many state-action pairs never visited.

Recap (Toward Monte Carlo Control)

Goal: Learn **action values** $q_\pi(s, a)$ so we can improve policies. (State values alone are not enough unless you have the full environment model, which we assume is not the case here.)

Exploration problem: Deterministic policies only visit a limited set of state–action pairs → cannot learn others. Solution: ensure all actions have non-zero probability.

Two exploration solutions

Exploring Starts (theoretical)

Start episodes from random (state, action) pair. Any state and action can be picked.
Guarantees exploring all state–action pairs.
Rarely practical. (In our food truck example we cannot start in arbitrary state, it must be (Monday,0)!)

Soft / ε-greedy Policies (practical)

With probability ε → choose random action (exploration).
With probability 1−ε → choose greedy action (exploitation).
Ensures continuous exploration inside the episode.

On-policy vs Off-policy MC

On-policy: learn value of the *same policy* used to generate data

Includes the negative effects of exploration (e.g., occasional bad actions).

Off-policy: learn value of a *different policy* (the target policy)

Can learn optimal greedy policy while still exploring.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Implementation: On-policy Monte Carlo Control

- See '**Solving_RL.ipynb**' file in the Teams course page.

- The Generalized Policy Iteration (GPI) framework suggests going back and forth between some forms of policy evaluation and policy improvement.
- We can use GPI with the Monte Carlo methods to obtain optimal policies.
- In each cycle, we collect a trajectory from a complete episode, then estimate the action-values and update the policy, and so on…
- Let's implement an on-policy MC control algorithm for the food truck inventory replenishment problem.

Implementation: On-policy Monte Carlo Control

- First, create a function that generates a random policy where all actions are equally likely to be taken, to be used to initialize the policy.

```
def get_random_policy(states, actions):
    policy = {}
    n_a = len(actions)
    for s in states:
        policy[s] = {a: 1/n_a for a in actions}
    return policy
```

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Implementation: On-policy Monte Carlo Control

- Then, build the on-policy first-visit MC control algorithm.

```python
def on_policy_first_visit_mc(env, n_iter, eps, gamma):
    np.random.seed(0)
    states =  env.state_space
    actions = env.actions_list
    policy =  get_random_policy(states, actions)
    Q = {s: {a: 0 for a in actions} for s in states}
    Q_n = {s: {a: 0 for a in actions} for s in states}
    for i in range(n_iter):
        if i % 10000 == 0:
            print("Iteration:", i)
        trajectory = get_trajectory(env, policy)
        G = 0
        T = len(trajectory) - 1
        for t, sar in enumerate(reversed(trajectory)):
            s, a, r = sar
            G = r + gamma * G
            first_visit = True
            for j in range(T - t):
                s_j = trajectory[j][0]
                a_j = trajectory[j][1]
                if (s, a) == (s_j, a_j):
                    first_visit = False
            if first_visit:
                Q[s][a] = Q_n[s][a] * Q[s][a] + G
                Q_n[s][a] += 1
                Q[s][a] /= Q_n[s][a]
                a_best = max(Q[s].items(), key=operator.itemgetter(1))[0]
                policy[s] = get_eps_greedy(actions, eps, a_best)
    return policy, Q, Q_n
```

Implementation: On-policy Monte Carlo Control

This is very similar to the first-visit MC prediction method, with the following key differences:

a) Instead of estimating the state values, $v(s)$, we estimate the action values, $q(s, a)$.

b) To explore all state-action pairs, we use ε-greedy policies.

c) This is a first-visit approach, but instead of checking if the state appeared earlier in the trajectory, we check if the state-action pair appeared before in the trajectory before we update the $q(s, a)$ estimate.

d) Whenever we update the $q(s, a)$ estimate of a state-action pair, we also update the policy to assign the highest probability to the action that maximizes the $q(s, a)$, which is $\underset{a}{\operatorname{argmax}}\, q(s, a)$.

Implementation: On-policy Monte Carlo Control

- Finally, use the algorithm to optimize the food truck policy.

```
policy, Q, Q_n = on_policy_first_visit_mc(foodtruck, 300000, 0.05, 1)
```

```
Iteration: 0
Iteration: 10000
Iteration: 20000
Iteration: 30000
Iteration: 40000
```

```
policy
```

```
{('Mon', 0): {0: 0.01, 100: 0.01, 200: 0.01, 300: 0.01, 400: 0.96},
 ('Tue', 0): {0: 0.01, 100: 0.01, 200: 0.01, 300: 0.01, 400: 0.96},
 ('Tue', 100): {0: 0.01, 100: 0.01, 200: 0.01, 300: 0.96, 400: 0.01},
 ('Tue', 200): {0: 0.01, 100: 0.01, 200: 0.96, 300: 0.01, 400: 0.01},
 ('Tue', 300): {0: 0.01, 100: 0.96, 200: 0.01, 300: 0.01, 400: 0.01},
 ('Wed', 0): {0: 0.01, 100: 0.01, 200: 0.01, 300: 0.01, 400: 0.96},
 ('Wed', 100): {0: 0.01, 100: 0.01, 200: 0.01, 300: 0.96, 400: 0.01},
 ('Wed', 200): {0: 0.01, 100: 0.01, 200: 0.96, 300: 0.01, 400: 0.01},
 ('Wed', 300): {0: 0.01, 100: 0.96, 200: 0.01, 300: 0.01, 400: 0.01},
 ('Thu', 0): {0: 0.01, 100: 0.01, 200: 0.01, 300: 0.96, 400: 0.01},
 ('Thu', 100): {0: 0.01, 100: 0.01, 200: 0.96, 300: 0.01, 400: 0.01},
 ('Thu', 200): {0: 0.01, 100: 0.96, 200: 0.01, 300: 0.01, 400: 0.01},
 ('Thu', 300): {0: 0.96, 100: 0.01, 200: 0.01, 300: 0.01, 400: 0.01},
 ('Fri', 0): {0: 0.01, 100: 0.01, 200: 0.96, 300: 0.01, 400: 0.01},
 ('Fri', 100): {0: 0.01, 100: 0.96, 200: 0.01, 300: 0.01, 400: 0.01},
 ('Fri', 200): {0: 0.96, 100: 0.01, 200: 0.01, 300: 0.01, 400: 0.01},
 ('Fri', 300): {0: 0.96, 100: 0.01, 200: 0.01, 300: 0.01, 400: 0.01},
 ('Weekend', 0): {0: 0.2, 100: 0.2, 200: 0.2, 300: 0.2, 400: 0.2},
 ('Weekend', 100): {0: 0.2, 100: 0.2, 200: 0.2, 300: 0.2, 400: 0.2},
 ('Weekend', 200): {0: 0.2, 100: 0.2, 200: 0.2, 300: 0.2, 400: 0.2},
 ('Weekend', 300): {0: 0.2, 100: 0.2, 200: 0.2, 300: 0.2, 400: 0.2}}
```

Cont
Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Implementation: Off-policy Monte Carlo Control

- In an off-policy approach, we have a set of samples (trajectories) **collected under some behavior policy b**, yet we would like to **use that experience to estimate the state and action values under a target policy π** .

- **How to estimate** the value of a target policy using data collected from a **different** behavior **policy**?
  Answer: *Importance sampling*.

Importance Sampling

- Let's start describing importance sampling in a simple game setting, in which you roll a six-sided die.
- Depending on what comes up on the die, you receive a random reward.
- This could be something like this: If you get a 1, you get a reward from a normal distribution with mean 10 and variance 25.
- There are similar hidden reward distributions for all outcomes, which are unknown to you.
- Let's denote the reward you receive when the face x comes up with the random variable $H(x)$, $x \in \{1, \ldots 6\}$.

Importance Sampling

- You want to estimate the expected reward after a single roll, which is:

$$\mathrm{E[H]} = \sum_{\mathrm{x=1}}^{6} \mathrm{E[H(x)]p(x)}$$

where $\mathrm{p(x)}$ is the probability of observing side x.

- Now, assume that you have two dice to choose from, A and B, with different probability distributions. You first pick A, and roll the die nn times. Your observations are as in the following table:

i	1	2	3	4	5	...	n
x_i	2	6	6	1	3	...	4
$h_i(x_i)$	10	23	0	94	21	...	32

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Importance Sampling

- Here $h_i(x_i)$ denotes the reward observed after the ith roll when the observed side is x_i. The estimated expected reward with die A, $\hat{\mu}_H^A$, is simply given by the following:

$$\hat{\mu}_H^A = \frac{1}{n}\sum_{x=1}^{n} h_i(x_i)$$

- **Importance sampling** suggests scaling these weights according to p_B:

$$\hat{\mu}_H^B = \frac{1}{n}\sum_{x=1}^{n} \frac{p_B(x_i)}{p_A(x_i)} h_i(x_i)$$

- Here the ratio $\rho(x) = \frac{p_B(x)}{p_A(x)} = \frac{p_{new}(x)}{p_{old}(x)}$ is called the **importance sampling ratio**.
- **Weighted importance sampling**:

$$\hat{\mu}_H^B = \frac{\sum_{i=1}^{n} \rho_{x_i} h_i(x_i)}{\sum_{i=1}^{n} \rho_{x_i}}$$

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Importance Sampling

- In the context of Monte Carlo prediction, observing a particular side of the die corresponds to **observing a particular trajectory**, and the die reward corresponds to the **total return**.
- Starting in some s_t, the probability of observing a trajectory $a_t, s_{t+1}, a_{t+1}, \dots, s_T$ under a behavior policy b is as follows:

$$\Pr\{a_t, s_{t+1}, a_{t+1}, \dots, s_T | s_t; a_{t:T-1} \sim b\}$$
$$= b(a_t|s_t) \cdot p(s_{t+1}|s_t, a_t) \cdot b(a_{t+1}|s_{t+1}) \cdot p(s_{t+2}|s_{t+1}, a_{t+1})$$
$$\cdot \cdots p(s_T|s_{T-1}, a_{T-1}) = \prod_{k=t}^{T-1} b(a_k|s_k) \cdot p(s_{k+1}|s_k, a_k)$$

- The expression is the same for the **target policy**, except $\boldsymbol{\pi}$ **replaces** $\boldsymbol{b}$.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Importance Sampling

- When we calculate the importance sampling ratio, the transition probabilities cancel out:

$$\rho_{t:T-1} = \prod_{k=t}^{T-1} \frac{\pi(a_k|s_k)}{b(a_k|s_k)}$$

- With that, we go from estimating the expectation under the behavior policy:

$$E[G_t|s] = v_b(s)$$

to estimating the expectation under the target policy:

$$E[\rho_{t:T-1} G_t|s] = v_\pi(s)$$

- Since we would not want to impose what a could be taken under the target policy, $\boldsymbol{b}$ **is usually** chosen as a **soft policy**.
- The ordinary importance sampling is unbiased but can have very high variance. The **weighted importance sampling**, on the other hand, is biased but usually has a much lower variance, hence it's **preferred in practice**.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Implementation: Off-policy Monte Carlo Control

- In our application of the off-policy Monte Carlo method, we use the weighted importance sampling.
- The behavior policy is chosen as an ϵ -greedy policy, yet the target is a greedy policy maximizing the action value for each state.
- We use an **incremental method** (online update) to update the state and action value estimates as follows (Sutton & Barto, 2018):

Implementation: Off-policy Monte Carlo Control

```
For each trajectory generated:
    G = 0
    W = 1
    for t from T-1 down to 0: # backward processing
      s, a, r = episode[t]

      G = gamma * G + r
      C[s][a] += W
      Q[s][a] += (W / C[s][a]) * (G - Q[s][a])

      if action taken by behavioral policy != action under target policy:
          break  # importance sampling ratio becomes zero, early states are almost never updated

      W = W * ( π(a|s) / b(a|s) ) # π(a|s) is 1 because of the greedy (deterministic) target policy
```

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Implementation: Off-policy Monte Carlo Control

```python
def off_policy_mc(env, n_iter, eps, gamma):
    np.random.seed(0)
    states =  env.state_space
    actions = env.actions_list
    Q = {s: {a: 0 for a in actions} for s in states}
    C = {s: {a: 0 for a in actions} for s in states}
    target_policy = {}
    behavior_policy = get_random_policy(states,
                                        actions)
    for i in range(n_iter):
        if i % 10000 == 0:
            print("Iteration:", i)
        trajectory = get_trajectory(env,
                                    behavior_policy)
        G = 0
        W = 1
        T = len(trajectory) - 1
        for t, sar in enumerate(reversed(trajectory)):
            s, a, r = sar
```

```python
        for t, sar in enumerate(reversed(trajectory)):
            s, a, r = sar
            G = r + gamma * G
            C[s][a] += W
            Q[s][a] += (W/C[s][a]) * (G - Q[s][a])
            a_best = max(Q[s].items(),
                         key=operator.itemgetter(1))[0]
            target_policy[s] = a_best
            behavior_policy[s] = get_eps_greedy(actions,
                                                eps,
                                                a_best)
            if a != target_policy[s]:
                break
            W = W / behavior_policy[s][a]
    target_policy = {s: target_policy[s] for s in states
                                 if s in target_policy}
    return target_policy, Q
```

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Implementation: Off-policy Monte Carlo Control

```
policy, Q = off_policy_mc(foodtruck, 300000, 0.05, 1)
```

```
Iteration: 0
Iteration: 10000
Iteration: 20000
Iteration: 30000
Iteration: 40000
```

```
policy
```

```
{('Mon', 0): 400,
 ('Tue', 0): 400,
 ('Tue', 100): 300,
 ('Tue', 200): 200,
 ('Tue', 300): 100,
 ('Wed', 0): 400,
 ('Wed', 100): 300,
 ('Wed', 200): 200,
 ('Wed', 300): 100,
 ('Thu', 0): 300,
 ('Thu', 100): 200,
 ('Thu', 200): 100,
 ('Thu', 300): 0,
 ('Fri', 0): 200,
 ('Fri', 100): 100,
 ('Fri', 200): 0,
 ('Fri', 300): 0}
```

Temporal Difference Learning

- MC methods can learn from experience but needs sampling complete episodes before working. This is not practicle.
- **Temporal-difference (TD)** methods are, in some sense, the best of dynamic programing and MC: They learn from experience, and they can update the policy after each step by **bootstrapping (do estimate based on other estimations)**.

	Dynamic Programming	Monte Carlo Methods	TD Learning
Learns from experience, without knowing environment dynamics	X	✓	✓
Uses bootstrapping	✓	X	✓
Updates policies with partial episode trajectories		X	✓

Table 5.2 – Comparison of DP, MC, and TD learning methods

Learning with Python, Enes Bilgin, Packt Publishing, 2020."

One-step TD learning – TD(0)

- TD methods can update a policy after a single state transition or multiple ones. The former version is called one-step TD learning or TD(0) and it is easier to implement compared to n -step TD learning.
- We'll start covering one-step TD learning with the **prediction problem**.
- We'll then introduce an **on-policy** method, **SARSA**, and then an **off-policy** algorithm, the famous **Q-learning**.

TD Prediction

- *The idea in TD learning is that we use this observation to update the existing estimate by moving it in the direction of this new estimate.*
- And the step-size $\boldsymbol{\alpha} \in \mathbf{(0,1]}$ controls how aggressively we move towards the new estimate.

- More formally, we use the following update rule:

$$\hat{\mathrm{v}}_\pi(\mathrm{s}) \coloneqq (1-\alpha)\hat{\mathrm{v}}_\pi(\mathrm{s}) + \alpha(\mathrm{r} + \gamma \mathrm{v}_\pi(s')) = \hat{\mathrm{v}}_\pi(\mathrm{s}) + \alpha[\mathrm{r} + \gamma \mathrm{v}_\pi(s') - \hat{\mathrm{v}}_\pi(\mathrm{s})]$$

- $\mathrm{r} + \gamma \mathrm{v}_\pi(s')$ is the new estimate for $\mathrm{v}_\pi(\mathrm{s})$ based on a single sample.
- The term in the square brackets is called the **TD error.**

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

TD Prediction

- α = 0 would completely ignore the new signal while α = 1 would completely ignore the existing estimate.
- Since a single observation is often noisy, and the new estimate itself uses another erroneous estimate ($v_\pi(s_{t+1})$); the new estimate cannot be solely relied on. Therefore, the α value is chosen between 0 and 1, and often closer to 0 .

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

State-value Estimation using TD(0)

- This function simply simulates a given environment using a specified policy over a specified number of iterations. After each observation, it does a one-step TD update using the given α step size and the discount factor γ .

```python
def one_step_td_prediction(env, policy, gamma, alpha, n_iter):
    np.random.seed(0)
    states = env.state_space
    v = {s: 0 for s in states}
    observation, info = env.reset()
    s = env.state_space[observation]
    for i in range(n_iter):
        a = choose_action(s, policy)
        action_index = env.actions_list.index(a)
        observation, reward, terminated, truncated, info = env.step(action_index)
        s_next = env.state_space[observation]
        v[s] += alpha * (reward + gamma * v[s_next] - v[s])
        if terminated or truncated:
            observation, info = env.reset()
            s = env.state_space[observation]
        else:
            s = s_next
    return v
```

```
{('Mon', 0): 2507.0,
 ('Tue', 0): 1956.0
...
```

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

On-policy control with SARSA

- With slight additions and modifications to the TD(0), we can turn it into an optimal control algorithm:
  - Ensure that we always have a soft policy, such as ϵ -greedy, to try all actions for a given state over time.
  - Estimate the action values.
  - Improve the policy based on the action-value estimates.
- We will do all of these at each step and using the observations
- $S_t = s, A_t = a, R_{t+1} = r, S_{t+1} = s', A_{t+1} = a'$, hence the name **SARSA**. The update formula:

$$\hat{\mathrm{q}}_\pi(\mathrm{s},\mathrm{a}) := \hat{\mathrm{q}}_\pi(\mathrm{s},\mathrm{a}) + \alpha[\mathrm{r} + \gamma\hat{\mathrm{q}}_\pi(s',\mathrm{a}') - \hat{\mathrm{q}}_\pi(\mathrm{s},\mathrm{a})]$$

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Implementation: On-policy control with SARSA

```python
def sarsa(env, gamma, eps, alpha, n_iter):
    np.random.seed(0)
    states = env.state_space
    actions = env.actions_list
    Q = {s: {a: 0 for a in actions} for s in states}
    policy = get_random_policy(states, actions)
    observation, info = env.reset()
    s = env.state_space[observation]
    a = choose_action(s, policy)
    action_index = env.actions_list.index(a)
    for i in range(n_iter):
        if i % 100000 == 0:
            print("Iteration:", i)
        observation, reward, terminated, truncated, info = env.step(action_index)
        s_next = env.state_space[observation]
        a_best = max(Q[s_next].items(), key=operator.itemgetter(1))[0]
        policy[s_next] = get_eps_greedy(actions, eps, a_best)
        a_next = choose_action(s_next, policy)
        Q[s][a] += alpha * (reward + gamma * Q[s_next][a_next] - Q[s][a])
        if terminated or truncated:
            observation, info = env.reset()
            s = env.state_space[observation]
            a_best = max(Q[s].items(), key=operator.itemgetter(1))[0]
            policy[s] = get_eps_greedy(actions, eps, a_best)
            a = choose_action(s, policy)
            action_index = env.actions_list.index(a)
        else:
            s = s_next
            a = a_next
    return policy, Q
```

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Implementation: On-policy control with SARSA

- Note that the exploratory actions would be ignored when implementing this policy after training, and we would simply always pick the action with the highest probability for each state.
- Notice that, however, the policy we obtain is a near-optimal one, **not the optimal one** we obtained with the DP methods.
- SARSA converges to an optimal solution in the limit, such as when n_iter → ∞ . In practice, we run the algorithm for a limited number of steps. Try increasing n_iter and you will see that the policy (usually) will get better.
- The learning rate α is a hyperparameter that needs to be tuned. The speed of the convergence depends on this selection.
- This is an on-policy algorithm. Therefore, the action values reflect the exploration due to the ϵ -greedy policy, which is not what we really want in this example.

Off-policy control with Q-learning

- We would like to isolate the action-value estimates from the exploration effect, which means having an off-policy method.
- Q-learning is such an approach, which makes it very powerful, and as a result, a very popular one.
- Here are how the action values are updated in Q-learning:

$$\hat{q}_\pi(s,a) := \hat{q}_\pi(s,a) + \alpha[r + \gamma \max_u \hat{q}_\pi(s',u) - \hat{q}_\pi(s,a)]$$

- Notice the max term after γ. The difference (with Sarsa) may look small, but it is key.
- *It means that the action the agent uses to update the action-value, u, is not necessarily the one it will take in the next step when in $S_{t+1} = s'$, $A_{t+1} = a'$. Instead, u is an action that maximizes $\hat{q}_\pi(s',\cdot)$, just like what we would use if not in training.*
- As a result, no exploratory actions are involved in action-value estimates and they are aligned with the policy that would be followed after training.

Off-policy control with Q-learning

- It means the action the agent takes in the next step, such as $\mathrm{A}_{t+1} = \mathrm{a}'$, is not used in the update.
- Instead, we use the maximum action value for the state $S_{t+1} = s'$, such as $\max_{\mathrm{u}} \hat{\mathrm{q}}_{\pi}(s', \mathrm{u})$, in the update.
- Such an action u is what we would use after training with those action values, hence no exploratory actions are involved in action-value estimations.

Implementation: Off-policy control with Q-learning

- The implementation of Q-learning is only slightly different than that of SARSA:

```
def q_learning(env, gamma, eps, alpha, n_iter):
    np.random.seed(0)
    states =  env.state_space
    actions = env.actions_list
    Q = {s: {a: 0 for a in actions} for s in states}
    policy = get_random_policy(states, actions)
    observation, info = env.reset()
    s = env.state_space[observation]
    for i in range(n_iter):
        if i % 100000 == 0:
            print("Iteration:", i)
        a_best = max(Q[s].items(), key=operator.itemgetter(1))[0]
        policy[s] = get_eps_greedy(actions, eps, a_best)
        a = choose_action(s, policy)
        action_index = env.actions_list.index(a)
        observation, reward, terminated, truncated, info = env.step(action_index)
        s_next = env.state_space[observation]
        Q[s][a] += alpha * (reward + gamma * max(Q[s_next].values()) - Q[s][a])
        if terminated or truncated:
            observation, info = env.reset()
            s = env.state_space[observation]
        else:
            s = s_next
    policy = {s: {max(policy[s].items(), key=operator.itemgetter(1))[0]: 1}for s in states}
    return policy, Q
```

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Implementation: Off-policy control with Q-learning

```
policy, Q = q_learning(foodtruck, 1, 0.1, 0.01, 1000000)
policy
```

```
Iteration: 0
Iteration: 100000
Iteration: 200000
Iteration: 300000
Iteration: 400000
Iteration: 500000
Iteration: 600000
Iteration: 700000
Iteration: 800000
Iteration: 900000
{('Mon', 0): {400: 1},
 ('Tue', 0): {400: 1},
 ('Tue', 100): {300: 1},
 ('Tue', 200): {200: 1},
 ('Tue', 300): {100: 1},
 ('Wed', 0): {400: 1},
 ('Wed', 100): {300: 1},
 ('Wed', 200): {200: 1},
 ('Wed', 300): {100: 1},
 ('Thu', 0): {300: 1},
 ('Thu', 100): {200: 1},
 ('Thu', 200): {100: 1},
 ('Thu', 300): {0: 1},
 ('Fri', 0): {200: 1},
 ('Fri', 100): {100: 1},
 ('Fri', 200): {0: 1},
 ('Fri', 300): {0: 1},
 ('Weekend', 0): {0: 1},
 ('Weekend', 100): {0: 1},
 ('Weekend', 200): {0: 1},
 ('Weekend', 300): {0: 1}}
```

You will see that this hyperparameter set gives you the optimal policy (or something close
to it depending on how randomization plays out in your case).

n-step TD learning

- **Key idea:**
  Monte Carlo waits until the **end of the episode** to update.
  TD(0) updates after **one step**.
  n-Step TD takes the middle path.

Example: 2-Step Return

$$G_t^{(2)} = R_{t+1} + \gamma R_{t+2} + \gamma^2 V(S_{t+2})$$

General n-Step Return

$$G_t^{(n)} = \sum_{k=1}^{n} \gamma^{k-1} R_{t+k} + \gamma^n V(S_{t+n})$$

- This form can be used in the TD update to reduce the weight of the estimates used in bootstrapping, which could be especially inaccurate at the beginning of the training.
- We don't include the implementation here as it gets a bit messy but still wanted to bring this alternative to your attention for you to have it in your toolkit.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."